SIMATS ENGINEERING

ASSESSMENT TOOL 3

ERROR ANALYSIS TASK

COURSE CODE: CSA3713

COURSE NAME: SOFTWARE TESTING

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow. (BL3)

Assessment Tool Weightage: 20%

Duration: 60 Minutes | On Class | Questions: 5 | Total Marks: 50

ANNEXURE A — ERROR ANALYSIS TASK

Code of Conduct

I Sameer Ahmed G / 192421154 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sameer Ahmed G

Q1. Error Analysis of an Online Examination System

Tools: Selenium WebDriver, Chrome DevTools, Excel

a) Analyze the reported errors: expected versus actual behavior

The reported symptoms indicate defects in eligibility validation or in the control flow between registration, eligibility checking, and examination access. The system must apply the same approved rule consistently to every student: a student may access the examination only when registration and register-number validation are successful, attendance is at least 75%, and the examination fee is paid.

Condition	Expected behavior	Reported / possible actual behavior	Likely area to investigate
Attendance < 75%	Reject examination access and show attendance-shortage message.	Student is granted access.	Boundary operator may use < 74 rather than < 75; eligibility check may be skipped or stale data used.
Attendance = 75%	Treat as eligible for the attendance rule, if the requirement says "at least 75%".	Eligible student is rejected.	Incorrect <= 75 condition, rounding issue, or attendance value mapped incorrectly.
Attendance > 75%	Pass the attendance condition.	Eligible student is rejected.	Wrong field, missing registration/fee status, or an unrelated validation error.
Fee unpaid / course not registered / invalid register number	Request payment or reject with the relevant validation message.	Access may be wrongly granted or an unclear message shown.	Incorrect rule ordering, missing validation, or poor error mapping.

Root-cause analysis should inspect the attendance source, numeric precision and rounding, comparison operators, registration and payment flags, session/cached data, and the order in which rules execute. Use DevTools Network to check the eligibility response and Console to locate client-side errors. The test oracle is the approved requirement, not merely the application's existing behavior.

b) Apply ECP, BVA, and Random Testing

Equivalence Class Partitioning (ECP) divides inputs into groups expected to behave similarly. Representative classes for attendance are: below 75% but within the valid range (0–74.99%), eligible attendance (75–100%), and invalid values below 0 or above 100. Registration, fee payment, and register number must also be partitioned into valid and invalid classes.

Technique / input	Test values	Expected result
ECP — attendance ineligible	0%, 50%, 74%	Reject access due to attendance shortage.
ECP — attendance eligible	75%, 82%, 100%	Pass attendance check; final access still requires all other conditions.
ECP — invalid attendance	−1%, 101%, blank, text	Validation error; no access.
BVA around threshold	74%, 75%, 76%	74% rejected; 75% and 76% pass attendance condition.
BVA at valid range limits	0%, 1%, 99%, 100%	Accept the range values for further eligibility checks.
Random testing	Random valid/invalid attendance, registration flag, payment flag, and register-number formats	No crash, bypass, inconsistent message, or unauthorized access.

ECP reduces repeated tests by selecting representative values from each class. BVA targets off-by-one defects near thresholds. Random tests add less predictable combinations, including decimals, blank fields, extreme numbers, whitespace, and malformed register numbers. Random results must be reproducible by recording the seed or exact generated inputs.

c) Decision-table and state-transition testing

The decision table below assumes an "AND" eligibility policy. If more than one condition fails, the product owner must define which error message takes precedence; each individual condition must still be tested independently.

Rule	Registered?	Attendance ≥75%?	Fee paid?	Register no. valid?	Expected outcome
R1	Yes	Yes	Yes	Yes	Grant access to examination.
R2	No	Any	Any	Any	Reject: course registration required.
R3	Yes	No	Any	Any	Reject: attendance shortage.
R4	Yes	Yes	No	Any	Request examination-fee payment.
R5	Yes	Yes	Yes	No	Reject and display register-number validation error.
R6	Yes	Yes	Yes	Yes	Grant access; verify that only one valid session/attempt is created.

Derived tests should cover each rule and each independent failing condition. Also test simultaneous failures to verify the documented priority of messages. A state model is: Not Registered → Registration Complete → Eligibility Check → (Rejected / Payment Required / Eligible) → Exam Access Granted → Exam Submitted. Invalid transitions include reaching Exam Access Granted from Attendance Shortage, Fee Unpaid, or Invalid Register Number.

```
Eligibility state flow:
Registration Complete → Eligibility Check
  ├─ invalid register number → Validation Error
  ├─ attendance < 75% → Rejected
  ├─ fee unpaid → Payment Required
  ├─ course not registered → Rejected
  └─ all conditions valid → Exam Access Granted
```

d) Selenium execution and error documentation

Use stable locators, explicit waits, a test account for each condition, and screenshots for failures. Replace the example URL and element locators with values from the authorized test environment. Never test against live student records without permission.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)
try:
    driver.get("https://TEST-ENVIRONMENT.example/login")  # replace with approved URL
    wait.until(EC.visibility_of_element_located((By.ID, "username"))).send_keys("TEST_STUDENT")
    driver.find_element(By.ID, "password").send_keys("TEST_PASSWORD")
    driver.find_element(By.ID, "loginButton").click()
    wait.until(EC.element_to_be_clickable((By.ID, "examRegistration"))).click()
    # Select a prepared test student with attendance below 75%.
    # Assert that the access/grant control is unavailable and shortage is shown.
    wait.until(EC.visibility_of_element_located((By.ID, "attendanceError")))
except Exception:
    driver.save_screenshot("Q1_examination_error.png")
    raise
finally:
    driver.quit()
```

For each run, record the test ID, user/data setup, browser version, steps, expected result, actual result, status, screenshot filename, and defect ID. A failed assertion is evidence of a defect only after the test data and requirement are confirmed.

e) Test adequacy and risk priorities

Measure execution and pass rates, but do not equate a high pass percentage with quality. Trace each rule to at least one positive and negative test. Highest priority should be given to unauthorized examination access, rejection of eligible students, incorrect fee/registration status, and session or direct-URL bypasses. Add tests for stale attendance data, decimal values around 75%, concurrent status changes, expired sessions, back-button navigation, and direct access to the exam URL. Release should be considered only after critical defects are closed and requirements coverage is reviewed.

Q2. Error Analysis of a Student Grade Calculator

Tools: Java, JUnit, Maven, JaCoCo

a) Source-code analysis: errors and root causes

The prompt does not include the faulty source listing, so the analysis below uses the common grading rule as an explicit reference specification: marks outside 0–100 are invalid; 90–100 = A; 75–89 = B; 50–74 = C; 0–49 = Fail. Confirm these thresholds with the approved course policy before implementation.

Potential defect	Root cause	Correction / justification
Boundary mark receives the wrong grade	Using > instead of ≥, or ordering checks incorrectly.	Use inclusive threshold checks in descending order; test 49/50, 74/75, and 89/90.
Negative mark or mark above 100 receives a grade	Missing range validation or invalid validation order.	Validate 0 ≤ mark ≤ 100 before grade assignment.
A lower grade masks a higher grade	Checking mark ≥50 before mark ≥90.	Evaluate thresholds from highest to lowest.
Wrong output for non-integer or missing input	No type/null validation or unhandled conversion exception.	Define accepted input type and return a documented error for unsupported values.
Test suite misses logical errors	Only typical values are tested; branches are not asserted.	Add boundary, invalid, branch, and mutation-focused tests.

b) ECP and Boundary Value Analysis

Input	Class / boundary	Expected grade
−1	Invalid: below minimum	Invalid
0	Lowest valid mark	Fail
49	Just below pass threshold	Fail
50	Pass threshold	C
74	Just below B threshold	C
75	B threshold	B
89	Just below A threshold	B
90	A threshold	A
100	Highest valid mark	A
101	Invalid: above maximum	Invalid

ECP uses one or more representatives from each grade interval and the two invalid ranges. BVA is essential because errors commonly occur at the exact transition values. For each threshold T, test T−1, T, and T+1 where those values remain in the valid domain.

c) CFG, independent paths, and cyclomatic complexity

```
START
  ↓
mark < 0 OR mark > 100? ──Yes──> return Invalid
  │ No
  ↓
mark ≥ 90? ─Yes─> return A
  │ No
  ↓
mark ≥ 75? ─Yes─> return B
  │ No
  ↓
mark ≥ 50? ─Yes─> return C
  │ No
  ↓
return Fail → STOP
```

Treating each if/else-if group as one decision, there are four decision nodes. Therefore, V(G) = number of decision nodes + 1 = 4 + 1 = 5. The equivalent graph calculation should also give V(G) = E − N + 2P = 5 for a single connected component, when nodes and edges are counted consistently. If the short-circuit OR expression is split into two separate decision nodes in the CFG, its detailed complexity count can differ; state the graph convention used.

Independent path	Input example	Outcome
P1: invalid range → return	−1 or 101	Invalid
P2: valid and mark ≥90	90	A
P3: <90 and ≥75	75	B
P4: <75 and ≥50	50	C
P5: <50	49	Fail

d) JUnit tests and JaCoCo coverage

```
public class ResultCalculator {
    public static String calculateGrade(int mark) {
        if (mark < 0 || mark > 100) return "Invalid";
        if (mark >= 90) return "A";
        if (mark >= 75) return "B";
        if (mark >= 50) return "C";
        return "Fail";
    }
}
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class ResultCalculatorTest {
    @Test void testsBoundariesAndInvalidValues() {
        assertEquals("Invalid", ResultCalculator.calculateGrade(-1));
        assertEquals("Fail", ResultCalculator.calculateGrade(0));
        assertEquals("Fail", ResultCalculator.calculateGrade(49));
        assertEquals("C", ResultCalculator.calculateGrade(50));
        assertEquals("C", ResultCalculator.calculateGrade(74));
        assertEquals("B", ResultCalculator.calculateGrade(75));
        assertEquals("B", ResultCalculator.calculateGrade(89));
        assertEquals("A", ResultCalculator.calculateGrade(90));
        assertEquals("A", ResultCalculator.calculateGrade(100));
        assertEquals("Invalid", ResultCalculator.calculateGrade(101));
    }
}
```

In a Maven project with the JaCoCo plugin configured, run `mvn test` and `mvn verify`, then open the generated JaCoCo HTML report (commonly under target/site/jacoco/index.html). Statement/line coverage shows executed code; branch coverage shows executed true/false outcomes. Full statement coverage alone may still miss a threshold boundary or an incorrect operator.

e) Mutation testing and adequacy evaluation

Mutation	Test that should kill it	Why it matters
mark >= 90 changed to mark > 90	mark = 90	Detects boundary operator error.
mark >= 75 changed to mark > 75	mark = 75	Detects wrong B threshold.
mark >= 50 changed to mark > 50	mark = 50	Detects pass-boundary error.
mark > 100 changed to mark >= 100	mark = 100	Detects rejection of a valid maximum.
Remove invalid-range check	mark = −1 and 101	Detects missing range validation.

A strong suite should cover each output class, threshold boundaries, invalid inputs, and decision branches, and should fail when these deliberate mutations are introduced. A surviving mutation indicates a test gap or an equivalent mutation requiring review. Coverage is evidence of execution, not proof of correctness; the grade rules and expected results still need confirmation by the course owner.

Q3. Error Analysis of an Online Banking Application

Tools: Postman, Swagger/OpenAPI, Excel

a) Analyze requests, responses, and observed errors

Use only a mock, local, or explicitly authorized test API. Capture the endpoint, HTTP method, headers (redacting secrets), request body, response status, response body, transaction ID, response time, and resulting balances. Compare each response with the approved transaction rules, not only with the HTTP status.

Error category	Expected behavior	Possible failure / likely root cause
Validation error	Reject zero/negative amount, malformed fields, insufficient balance, or over-limit transfer with a consistent 4xx response.	Request accepted because validation is missing, uses a wrong boundary operator, or runs after processing.
Authentication error	Unauthenticated or expired-session requests must not transfer funds.	Missing auth middleware, token not checked for the endpoint, or expired token treated as valid.
Transaction consistency	Balance is reduced only once after a confirmed successful transfer.	Balance deducted on failure, duplicate processing, or non-atomic balance/ledger update.
Network failure	Return or expose a clear pending/failed state; balance and ledger remain consistent.	Client reports failure while server completes transfer, or retries create a duplicate.
Response inconsistency	Use documented status codes and a stable error schema.	Same error yields unrelated codes/messages or response fields differ between endpoints.

b) Apply black-box test design techniques

Technique	Representative test inputs	Expected result
ECP — amount	Negative; zero; positive within balance/limit; greater than balance; above daily limit; missing/string amount.	Invalid classes rejected; valid class proceeds to processing.
BVA — minimum positive amount	0, smallest allowed unit, just above minimum.	0 rejected; minimum permitted amount accepted if funds/limits allow.
BVA — available balance B	B−smallest unit, B, B+smallest unit.	At or below balance can pass balance validation; above balance rejected.
BVA — daily limit L	L−smallest unit, L, L+smallest unit.	Over-limit transfer rejected; exact-limit behavior follows approved policy.
Random/error guessing	Large magnitudes, decimals, null, negative values, malformed account IDs, extra fields, repeated identical request.	No crash, security bypass, inconsistent balance, or unhandled server error.

Decision-table testing should cover authenticated/unauthenticated, positive/invalid amount, sufficient/insufficient balance, within/over daily limit, and available/unavailable network. Test one failure condition at a time, then important combinations to verify error precedence and safe rejection.

c) Use-case, state-transition, and pairwise testing

A primary use case is: authenticate → retrieve balance → submit transfer → receive status → verify ledger and balance. Alternate flows include invalid credentials, expired session, invalid beneficiary,

insufficient funds, over-limit amount, network interruption, timeout, and retry. For a repeated request, verify idempotency: the same logical transfer must not be executed twice.

```
Transaction states:
Pending → Successful
Pending → Failed
Pending → Cancelled (only where cancellation is supported)
Successful must not transition to Failed merely because the client times out.
A retry should query/reuse the original transaction or use an idempotency key.
```

Test	Browser	Network	Authentication	Request pattern / expected behavior
P1	Chrome	Wi-Fi	Valid	First request; process at most one transfer.
P2	Chrome	Wi-Fi	Expired	Retry; reject without transferring funds.
P3	Chrome	Mobile data	Valid	Retry with same idempotency key; no duplicate.
P4	Chrome	Mobile data	Expired	First request; reject with documented auth error.
P5	Edge	Wi-Fi	Valid	Retry; return/reconcile original transaction.
P6	Edge	Wi-Fi	Expired	First request; reject without balance change.
P7	Edge	Mobile data	Valid	First request; verify status and ledger.
P8	Edge	Mobile data	Expired	Retry; reject safely and do not duplicate.

This eight-run suite covers every pair of values among four binary factors: browser, network, authentication state, and request pattern. Transaction status is an outcome modeled by the state transitions, not an independent input. Pairwise coverage reduces combinations, but targeted tests are still required for concurrent requests, network loss at commit time, and each Pending → Successful/Failed transition.

d) Postman execution and defect reports

Create a Postman collection with environment variables for base URL and test account IDs. Use separate requests for authentication, balance retrieval, valid transfer, invalid transfer, limit boundary, and duplicate request. Store actual status codes and balances before/after each request. A sample assertion pattern is:

```
pm.test("Response uses an approved status", function () {
  pm.expect([200, 201, 400, 401, 403, 409, 422]).to.include(pm.response.code);
});
pm.test("Response time is within agreed limit", function () {
  pm.expect(pm.response.responseTime).to.be.below(2000); // replace with approved SLA
});
// For a deliberately invalid transfer, assert the documented 4xx code and error schema.
// Never hard-code or publish real credentials or access tokens.
```

For each defect, include reproduction steps, sanitized request/response, status code, expected/actual result, transaction ID, before/after balance evidence, severity, priority, environment, and defect ID. Do not claim an error was observed until the request has actually been run.

e) Adequacy, traceability, and code-coverage limits

Trace every transaction rule to executed positive, negative, boundary, authentication, network, and repeated-request tests. Prioritize unauthorized or duplicate transfers, balance inconsistency, and silent

failure. Without source code, internal CFG, statement/branch coverage, cyclomatic complexity, and path coverage cannot normally be measured; API behavior, requirement/decision-table coverage, and transaction reconciliation remain assessable.

Q4. Error Analysis of a COTS Learning Management System

Tools: Selenium WebDriver, Apache JMeter, Chrome DevTools

a) Reverse engineer observable functionality and identify errors

Treat the LMS as a black box. Inspect available pages and navigation, record inputs/outputs, monitor network requests, and compare observable behavior with approved university requirements. Decompose the LMS into authentication, course enrollment, assignment upload, submission status, result viewing, session management, and reporting.

Function	Expected behavior	Error symptoms to investigate
Login	Valid user enters system; invalid credentials are rejected safely.	Wrong password accepted, correct user rejected, session not created/expired incorrectly.
Course enrollment	Eligible student can enroll; duplicates or unavailable courses are handled clearly.	Enrollment shown but not saved, duplicate enrollments, inconsistent capacity information.
Assignment upload	Allowed file type/size accepted; invalid file rejected with message; submission recorded.	Upload fails, file silently truncated, unsupported file accepted, status not persisted.
Result viewing	Authorized student sees only their own released results.	Wrong result displayed, unauthorized access, stale or inconsistent data.

b) ECP, BVA, exploratory testing, and error guessing

Let S be the configured maximum upload size and let F be an allowed file type. Do not invent an exact limit if the vendor or university has not supplied it; obtain the published requirement and test around it.

Test area	Test inputs / method	Expected result
File-size BVA	S−1 byte, S, S+1 byte; empty file; very large file.	Allowed files at/below limit accepted as specified; above limit rejected clearly.
File-type ECP	Supported type, unsupported extension, wrong MIME type, renamed executable, corrupted file.	Only approved types accepted; unsafe or inconsistent files rejected.
Login ECP/BVA	Valid/invalid credentials; blank fields; length boundaries and whitespace.	Correct authentication and useful validation feedback.
Enrollment decision rules	Eligible/ineligible student, open/full course, repeat enrollment.	Only permitted enrollment states are created.
Exploratory/error guessing	Refresh during upload, cancel, network switch, double-click submit, back button, session expiry.	No data loss, duplicate submission, leaked data, or stuck state.

c) Selenium compatibility and JMeter performance investigation

Use Selenium for repeatable browser workflows and verify behavior in every supported browser/OS/device combination. Use explicit waits rather than fixed sleeps; capture screenshots and browser/driver versions for failures. Use Chrome DevTools Network to check failed requests, status codes, console errors, upload request payload/size, caching, and timing.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 15)
try:
    driver.get("https://TEST-LMS.example/login")  # approved test environment
    wait.until(EC.visibility_of_element_located((By.ID, "username"))).send_keys("TEST_USER")
    driver.find_element(By.ID, "password").send_keys("TEST_PASSWORD")
    driver.find_element(By.ID, "loginButton").click()
    wait.until(EC.visibility_of_element_located((By.ID, "dashboard")))
    # Continue through course and assignment workflows using verified locators.
except Exception:
    driver.save_screenshot("LMS_workflow_failure.png")
    raise
finally:
    driver.quit()
```

In JMeter, define a Thread Group with a controlled ramp-up and duration, add requests to authorized endpoints, handle session correlation, and assert responses. Record median and 90th/95th percentile response time, throughput, error rate, and server saturation; compare these metrics with predefined acceptance thresholds. Run only in an approved staging environment under an agreed workload.

d) Limitations of COTS testing

Without vendor source code or suitable instrumentation, testers generally cannot reconstruct internal CFGs, enumerate source-level independent paths, calculate source statement/branch coverage, or run JaCoCo/Coverage.py on the product. Black-box testing can still assess visible behavior, API contracts, response time, compatibility, and security controls. Use vendor documentation and release notes as supporting evidence; do not infer internal coverage from UI test counts.

e) Test adequacy and acceptance recommendation

Quality dimension	Evidence	Suggested acceptance condition
Functionality	Requirement-to-test traceability; login, enrollment, upload, results workflows.	All critical requirements tested; no unresolved critical functional defect.
Compatibility	Results by supported browser, OS, and device.	All required combinations pass or have approved known limitations.
Performance	JMeter response-time percentiles, throughput, error rate under agreed load.	Metrics satisfy the university's documented SLA.
Security and access	Role checks, direct URL access, session expiry, file upload validation.	No unresolved critical/high risk that violates policy.
Usability / reliability	Exploratory findings, repeat submissions, interrupted uploads, recovery.	Critical workflows recover safely and messages are understandable.

Recommend acceptance only when the agreed thresholds are met and critical risks are controlled. Conditional acceptance requires documented limitations, vendor commitments, owners, and deadlines. Reject or defer deployment when high-impact upload, compatibility, security, or peak-load failures

remain unresolved. Since no live LMS results or acceptance thresholds were supplied, this is the decision framework rather than a claim that the product passed.

Q5. Error Analysis of a Student Result-Processing Program

Tools: Python, pytest, Coverage.py, Ruff or Pylint

a) Analyze conditions, loops, definitions, and uses

The scenario does not provide the faulty program listing or official limits. For the test design below, assume the requirements are: student count N is between 1 and MAX_STUDENTS; each student has M subjects between 1 and MAX_SUBJECTS; each mark is an integer from 0 to 100; the pass threshold is 50. These values should be replaced with approved course rules if they differ.

Possible error	Root cause	Corrective action
Total carries over from prior student	`total = 0` is initialized outside the outer loop.	Reset total at the start of each student iteration.
One extra subject is processed	Incorrect loop range or <= condition.	Use `range(M)` for exactly M subject entries and test 0/1/MAX/M+1 boundaries.
Invalid marks accepted	Missing range/type check or check applied after accumulation.	Validate each mark before adding it to total.
Incorrect zero-subject result	Zero subjects are not explicitly accepted or rejected.	Define policy; preferably reject M<1 unless zero-subject results are explicitly allowed.
Wrong pass/fail decision	Incorrect threshold operator or stale total/average.	Use the approved threshold and test values 49, 50, 51.
Max-count failure	Off-by-one range, memory/performance issue, or missing count validation.	Validate configured maxima and test at and just beyond each limit.

b) Boundary values, equivalence classes, and error guessing

Input	Representative test values	Expected result under stated assumptions
Student count N	0, 1, typical, MAX_STUDENTS, MAX_STUDENTS+1	Reject 0 and above max; accept permitted counts.
Subject count M	0, 1, typical, MAX_SUBJECTS, MAX_SUBJECTS+1	Reject 0 and above max; process exactly M subjects.
Mark	−1, 0, 49, 50, 51, 99, 100, 101	Reject outside 0–100; valid marks included once.
Data type	Blank, text, decimal if integers required, null/missing field	Clear validation error; no crash or partial result.
Loop / state	Two students: first marks total 150, second total 70; repeat with different counts.	Second result should be 70, not 220; totals remain independent.
Error guessing	Negative count, huge count, missing mark, duplicate record, interruption mid-input.	Reject invalid input safely; avoid partial/inconsistent results.

c) CFG, complexity, independent paths, and loop tests

```
START → read N → validate N
  ↓ valid
Outer loop: for each student i
  → total = 0 → read/validate M
  → Inner loop: for each subject j
       read mark → validate mark → total = total + mark
  → compute pass/fail → print total/result
  → next student or STOP
```

A flow graph should include initialization, validation branches, outer-loop condition, inner-loop condition, mark validation, accumulation, result decision, and exit. Cyclomatic complexity depends on

the exact implementation and the decision nodes used in its CFG. For a reference implementation containing five binary decisions (N valid, outer-loop condition, M valid, inner-loop condition, mark valid, plus a pass/fail decision) the count must be performed carefully: six binary decision nodes yield V(G) = 6 + 1 = 7. If the chosen implementation has a different number of conditions, recalculate from that graph using V(G)=E−N+2P and confirm with decisions + 1 for a single connected function.

Independent-path testing should exercise invalid N, valid N with invalid M, valid counts with invalid mark, inner loop repeated once and multiple times, outer loop with multiple students, and both pass/fail outcomes. Loop tests should include the minimum valid number of iterations, one iteration, typical iterations, maximum permitted iterations, and an over-maximum input rejected before the loop starts.

Test ID	Input summary	Expected output / behavior
LP1	N=0	Validation error; no result rows.
LP2	N=1, M=1, mark=50	One result; total=50; pass.
LP3	N=2; student totals 150 and 70	Two independent results; second total is 70.
LP4	M=MAX_SUBJECTS for one student	Exactly MAX_SUBJECTS marks are read and summed.
LP5	M=MAX_SUBJECTS+1	Reject before reading/processing marks.
LP6	A mark=−1 or 101	Validation error; invalid mark not added.
LP7	All marks below 50 vs total/average at threshold	Pass/fail follows the documented rule consistently.

d) pytest, Coverage.py, data-flow, and mutation testing

```
MAX_STUDENTS = 100       # illustrative; replace with approved maximum
MAX_SUBJECTS = 10        # illustrative; replace with approved maximum

def process_results(records):
    if not isinstance(records, list) or not (1 <= len(records) <= MAX_STUDENTS):
        raise ValueError("Invalid student count")
    output = []
    for student_marks in records:
        if not isinstance(student_marks, list) or not (1 <= len(student_marks) <=
MAX_SUBJECTS):
            raise ValueError("Invalid subject count")
        total = 0
        for mark in student_marks:
            if not isinstance(mark, int) or isinstance(mark, bool) or not (0 <= mark <= 100):
                raise ValueError("Invalid mark")
            total += mark
        output.append({"total": total, "passed": all(m >= 50 for m in student_marks)})
    return output
```

This is a reference design, not the missing original program. It assumes a student passes only if every subject mark is at least 50; if the institution uses average/overall-total rules, change that decision to match the approved specification.

```
import pytest
from result_processor import process_results

def test_totals_reset_for_each_student():
    result = process_results([[80, 70], [30, 40]])
    assert [x["total"] for x in result] == [150, 70]

def test_mark_boundaries():
    assert process_results([[0]])[0]["total"] == 0
    assert process_results([[100]])[0]["total"] == 100

@pytest.mark.parametrize("bad_mark", [-1, 101, "50", None])
def test_invalid_marks_are_rejected(bad_mark):
    with pytest.raises(ValueError):
        process_results([[bad_mark]])

def test_invalid_subject_count_is_rejected():
    with pytest.raises(ValueError):
        process_results([[]])
```

Run `pytest -q` and then `coverage run --branch -m pytest`; create a report with `coverage report -m` or `coverage html`. Branch measurement shows whether true/false outcomes executed. Data-flow testing traces definitions and uses: N is defined by input and used in range validation/loop control; M is defined per student and used to control the inner loop; mark is defined by each input and used for validation, total, and pass/fail; total is initialized per student, updated per valid mark, and used when output is created. A missing definition or stale value can produce incorrect totals.

Useful mutations include moving `total = 0` outside the outer loop; changing `range(M)` to `range(M + 1)`; changing mark validation from `mark > 100` to `mark >= 100`; changing `m >= 50` to `m > 50`; and removing the subject-count check. Targeted tests should fail for each mutation. A surviving mutation suggests missing assertions, inadequate boundary data, or equivalent behavior that should be examined.

e) Evaluate test adequacy and release readiness

Evaluate the final suite using several complementary measures: statement coverage (executed statements), branch coverage (true/false decision outcomes), loop coverage (minimum/one/typical/maximum repetitions), data-flow coverage (important definition-use paths), boundary coverage, and mutation score (non-equivalent mutants detected divided by non-equivalent mutants run). Report the numerator and denominator rather than a percentage alone.

A release decision should require all critical requirements to map to passing tests; no unresolved critical defect in validation, totals, or pass/fail logic; maximum-loop tests within acceptable runtime; invalid inputs rejected without partial/corrupt results; and mutation results reviewed. High statement coverage alone is insufficient because an incorrect boundary operator or stale `total` may execute fully

while still producing wrong outputs. Add tests for any uncovered branches, surviving non-equivalent mutations, and unresolved requirements before release.

Overall Conclusion

Error analysis combines symptom reproduction, expected-versus-actual comparison, root-cause reasoning, appropriate test design, correction verification, and evidence-based test adequacy. Black-box techniques expose incorrect behavior from the outside; white-box techniques help assess decision paths and internal logic when source code is available. Defect fixes should be verified with retesting and regression tests, while coverage, traceability, and mutation analysis provide complementary evidence for release readiness.